

Algorithms

Supantha Pandit

DA-IICT

Chapter Summary

- 1 Properties of Algorithms
- 2 The Growth of Functions
- 3 Complexity of Algorithms

Algorithms

Algorithm: An algorithm is a finite set of precise instructions to perform a computation or solve a problem.

Example: Describe an algorithm for finding the maximum value in a finite sequence of integers.

Description of algorithms in pseudocode:

- Intermediate step between English prose and formal coding in a programming language.
- Focus on the fundamental operation of the program, instead of peculiarities of a given programming language.
- Analyze the time required to solve a problem using an algorithm, independent of the actual programming language.

Properties of algorithms

- **Input:** An algorithm has input values from a specified set.
- **Output:** From the input values, the algorithm produces the output values from a specified set. The output values are the solution.
- **Correctness:** An algorithm should produce the correct output values for each set of input values.
- **Finiteness:** An algorithm should produce the output after a finite number of steps for any input.
- **Effectiveness:** It must be possible to perform each step of the algorithm correctly and in a finite amount of time.
- **Generality:** The algorithm should work for all problems of the desired form.

Example: maximum of three

Problem: Finds the maximum of three numbers a , b , and c :

A formal description: (steps in simple english)

- 1 $large = a$.
- 2 If $b > large$, then $large = b$.
- 3 If $c > large$, then $large = c$.

Pseudocode:

Input: a, b, c

Output: $large$ (the largest of a , b , and c)

Max-three(a, b, c)

$large = a$

if ($b > large$) // if b is larger than $large$, update $large$

$large = b$

if ($c > large$) // if c is larger than $large$, update $large$

$large = c$

return $large$

Maximum Value in a Sequence

Problem: Describe an algorithm for finding the maximum (largest) value in a finite sequence of integers.

A formal description:

- 1 Set the **temporary maximum** equal to the first integer in the sequence. (The **temporary maximum** will be the largest integer examined at any stage of the procedure.)
- 2 Compare the next integer in the sequence to the **temporary maximum**, and if it is larger than the **temporary maximum**, set the **temporary maximum** equal to this integer.
- 3 Repeat previous step if there are more integers in the sequence.
- 4 Stop when there are no integers left in the sequence. Finally, the **temporary maximum** is the largest integer in the sequence.

Maximum Value in a Sequence

Pseudocode:

This algorithm finds the largest of the numbers $A = a_1, a_2, \dots, a_n$.

Max($a_1, a_2, \dots, a_n$: integers)

Input: $a_1, a_2, \dots, a_n, n$

Output: *large* (the largest value in the sequence A)

Max($a_1, a_2, \dots, a_n, n$)

large = a_1

for $i = 2$ to n

 if ($a_i > large$)

$large = a_i$

return *large*

Integer multiplication

Problem: Given two n -digit integers x and y , calculate $x \times y$.

Primitive operations: Add or multiply two single-digit numbers.

What about integer addition:

Problem: Given two n -digit integers x and y , compute $x + y$.

Example: Find $1234 + 5678$

Integer multiplication

Problem: Given two n -digit integers x and y , calculate $x \times y$.

Primitive operations: Add or multiply two single-digit numbers.

What about integer addition:

Problem: Given two n -digit integers x and y , compute $x + y$.

Example: Find $1234 + 5678$

Grade school method:

$$\begin{array}{r} 011 \\ 1234 \\ + 5678 \\ \hline 6912 \end{array}$$

Integer multiplication

Problem: Given two n -digit integers x and y , calculate $x \times y$.

Primitive operations: Add or multiply two single-digit numbers.

What about integer addition:

Problem: Given two n -digit integers x and y , compute $x + y$.

Example: Find $1234 + 5678$

Grade school method:

$$\begin{array}{r} 011 \\ 1234 \\ + 5678 \\ \hline 6912 \end{array}$$

Question: How many bit operations are required???

Integer multiplication

Problem: Given two n -digit integers x and y , calculate $x \times y$.

Primitive operations: Add or multiply two single-digit numbers.

What about integer addition:

Problem: Given two n -digit integers x and y , compute $x + y$.

Example: Find $1234 + 5678$

Grade school method:

$$\begin{array}{r} 011 \\ 1234 \\ + 5678 \\ \hline 6912 \end{array}$$

Question: How many bit operations are required???

Answer: $\text{constant} \times n$

This is optimal

Integer multiplication

Example: Find 1234×5678

Integer multiplication

Example: Find 1234×5678

$$\begin{array}{r} 5\ 6\ 7\ 8 \\ \times 1\ 2\ 3\ 4 \\ \hline 2\ 2\ 7\ 1\ 2 \\ 1\ 7\ 0\ 3\ 4\ - \\ 1\ 1\ 3\ 5\ 6\ -\ - \\ 5\ 6\ 7\ 8\ -\ -\ - \\ \hline 7\ 0\ 0\ 6\ 6\ 5\ 2 \end{array}$$

Integer multiplication

Example: Find 1234×5678

Question: Compute the total primitive operations (in terms of n).

$$\begin{array}{r} 5 6 7 8 \\ \times 1 2 3 4 \\ \hline 2 2 7 1 2 \\ 1 7 0 3 4 - \\ 1 1 3 5 6 - - \\ 5 6 7 8 - - - \\ \hline 7 0 0 6 6 5 2 \end{array}$$

Integer multiplication

Example: Find 1234×5678

$$\begin{array}{r} 5 6 7 8 \\ \times 1 2 3 4 \\ \hline 2 2 7 1 2 \\ 1 7 0 3 4 - \\ 1 1 3 5 6 - - \\ 5 6 7 8 - - - \\ \hline 7 0 0 6 6 5 2 \end{array}$$

Question: Compute the total primitive operations (in terms of n).

Answer:

- Each row:

Integer multiplication

Example: Find 1234×5678

$$\begin{array}{r} 5 6 7 8 \\ \times 1 2 3 4 \\ \hline 2 2 7 1 2 \\ 1 7 0 3 4 - \\ 1 1 3 5 6 - - \\ 5 6 7 8 - - - \\ \hline 7 0 0 6 6 5 2 \end{array}$$

Question: Compute the total primitive operations (in terms of n).

Answer:

- Each row: $\text{constant} * n$
- All rows:

Integer multiplication

Example: Find 1234×5678

$$\begin{array}{r} 5\ 6\ 7\ 8 \\ \times 1\ 2\ 3\ 4 \\ \hline 2\ 2\ 7\ 1\ 2 \\ 1\ 7\ 0\ 3\ 4\ - \\ 1\ 1\ 3\ 5\ 6\ -\ - \\ 5\ 6\ 7\ 8\ -\ -\ - \\ \hline 7\ 0\ 0\ 6\ 6\ 5\ 2 \end{array}$$

Question: Compute the total primitive operations (in terms of n).

Answer:

- Each row: $\text{constant} * n$
- All rows: $n(\text{constant} * n)$
- Digits in each row is at most

Integer multiplication

Example: Find 1234×5678

$$\begin{array}{r} 5\ 6\ 7\ 8 \\ \times 1\ 2\ 3\ 4 \\ \hline 2\ 2\ 7\ 1\ 2 \\ 1\ 7\ 0\ 3\ 4\ - \\ 1\ 1\ 3\ 5\ 6\ -\ - \\ 5\ 6\ 7\ 8\ -\ -\ - \\ \hline 7\ 0\ 0\ 6\ 6\ 5\ 2 \end{array}$$

Question: Compute the total primitive operations (in terms of n).

Answer:

- Each row: $\text{constant} * n$
- All rows: $n(\text{constant} * n)$
- Digits in each row is at most $2n$.
- Sum two rows:

Integer multiplication

Example: Find 1234×5678

$$\begin{array}{r} 5 6 7 8 \\ \times 1 2 3 4 \\ \hline 2 2 7 1 2 \\ 1 7 0 3 4 - \\ 1 1 3 5 6 - - \\ 5 6 7 8 - - - \\ \hline 7 0 0 6 6 5 2 \end{array}$$

Question: Compute the total primitive operations (in terms of n).

Answer:

- Each row: $\text{constant} * n$
- All rows: $n(\text{constant} * n)$
- Digits in each row is at most $2n$.
- Sum two rows: $\text{constant} * n$
- Sum all rows:

Integer multiplication

Example: Find 1234×5678

$$\begin{array}{r} 5\ 6\ 7\ 8 \\ \times 1\ 2\ 3\ 4 \\ \hline 2\ 2\ 7\ 1\ 2 \\ 1\ 7\ 0\ 3\ 4\ - \\ 1\ 1\ 3\ 5\ 6\ -\ - \\ 5\ 6\ 7\ 8\ -\ -\ - \\ \hline 7\ 0\ 0\ 6\ 6\ 5\ 2 \end{array}$$

Question: Compute the total primitive operations (in terms of n).

Answer:

- Each row: $\text{constant} * n$
- All rows: $n(\text{constant} * n)$
- Digits in each row is at most $2n$.
- Sum two rows: $\text{constant} * n$
- Sum all rows:
 $(n - 1)(\text{constant} * n)$
- Total operations:

Integer multiplication

Example: Find 1234×5678

$$\begin{array}{r} 5\ 6\ 7\ 8 \\ \times 1\ 2\ 3\ 4 \\ \hline 2\ 2\ 7\ 1\ 2 \\ 1\ 7\ 0\ 3\ 4\ - \\ 1\ 1\ 3\ 5\ 6\ -\ - \\ 5\ 6\ 7\ 8\ -\ -\ - \\ \hline 7\ 0\ 0\ 6\ 6\ 5\ 2 \end{array}$$

Question: Compute the total primitive operations (in terms of n).

Answer:

- Each row: $\text{constant} * n$
- All rows: $n(\text{constant} * n)$
- Digits in each row is at most $2n$.
- Sum two rows: $\text{constant} * n$
- Sum all rows:
 $(n - 1)(\text{constant} * n)$
- Total operations: $\text{constant} * n^2$

The algorithm designer's mantra

Can we do better??

Than the “obvious” method.

Karatsuba multiplication

Example: $x = 5678$ and $y = 1234$

$$x = \underbrace{56}_a \underbrace{78}_b$$

and

$$y = \underbrace{12}_c \underbrace{34}_d$$

- **Step 1:** Compute $a \cdot c$
- **Step 2:** Compute $b \cdot d$
- **Step 3:** Compute $(a + b) \cdot (c + d)$
- **Step 4:** Step 3 - Step 2 - Step 1
- **Step 5:** $10^n ac + 10^{n/2} \cdot \text{Step 4} + bd$

Karatsuba multiplication

Example: $x = 5678$ and $y = 1234$

$$x = \underbrace{56}_a \underbrace{78}_b$$

and

$$y = \underbrace{12}_c \underbrace{34}_d$$

- **Step 1:** Compute $a \cdot c$
- **Step 2:** Compute $b \cdot d$
- **Step 3:** Compute $(a + b) \cdot (c + d)$
- **Step 4:** Step 3 - Step 2 - Step 1
- **Step 5:** $10^n ac + 10^{n/2} \cdot \text{Step 4} + bd$

$$x = 10^{n/2}a + b \quad y = 10^{n/2}c + d$$

$$x \times y = (10^{n/2}a + b) \times (10^{n/2}c + d)$$

$$x \times y = 10^a c + 10^{n/2}(ad + bc) + bd$$

$$x \times y = 10^a c + 10^{n/2}(ad + bc) + bd$$

$$x \times y = 10^a c + 10^{n/2}[(a + b)(c + d) - ac - bd] + bd$$

Time complexity

$a, b, c, d : \frac{n}{2}$ digit numbers

$T(n)$: Total time required by the algorithm.

Time complexity

$a, b, c, d : \frac{n}{2}$ digit numbers

$T(n)$: Total time required by the algorithm.

- **Step 1:** $T(n/2)$

Time complexity

$a, b, c, d : \frac{n}{2}$ digit numbers

$T(n)$: Total time required by the algorithm.

- **Step 1:** $T(n/2)$
- **Step 2:** $T(n/2)$

Time complexity

$a, b, c, d : \frac{n}{2}$ digit numbers

$T(n)$: Total time required by the algorithm.

- **Step 1:** $T(n/2)$
- **Step 2:** $T(n/2)$
- **Step 3:** $T(n/2)$

Time complexity

$a, b, c, d : \frac{n}{2}$ digit numbers

$T(n)$: Total time required by the algorithm.

- **Step 1:** $T(n/2)$
- **Step 2:** $T(n/2)$
- **Step 3:** $T(n/2)$
- **Step 4:** $\text{constant} \cdot n$

Time complexity

$a, b, c, d : \frac{n}{2}$ digit numbers

$T(n)$: Total time required by the algorithm.

- **Step 1:** $T(n/2)$
- **Step 2:** $T(n/2)$
- **Step 3:** $T(n/2)$
- **Step 4:** $\text{constant} \cdot n$
- **Step 5:** $\text{constant} \cdot n$

Time complexity

$a, b, c, d : \frac{n}{2}$ digit numbers

$T(n)$: Total time required by the algorithm.

- **Step 1:** $T(n/2)$
- **Step 2:** $T(n/2)$
- **Step 3:** $T(n/2)$
- **Step 4:** $\text{constant} \cdot n$
- **Step 5:** $\text{constant} \cdot n$

Total time: $T(n) = 3T(n/2) + \text{constant} \cdot n$

A question

Question: Which of the two algorithms is faster???

- $\text{constant} \cdot n^2$
- $T(n) = 3T(n/2) + \text{constant} \cdot n$

- How to compare (running time of) two algorithms.
- How to solve equations like $T(n) = 3T(n/2) + \text{constant} \cdot n$

Growth of functions

Asymptotic notations:

- Big “oh” notation O
- Small “oh” notation o
- Big “omega” notation Ω
- Small “omega” notation ω
- “theta” notation Θ

Big “oh” notation (upper bound)

Let $f(n)$ be a function defined on $n = 1, 2, 3, \dots$

$f(n) = O(g(n))$ if there exist constants $c > 0$, $n_0 > 0$ such that

$$0 \leq f(n) \leq cg(n) \quad \text{for all } n \geq n_0$$

Warning: c and n_0 are constants and do not depend on n .

Example: Is $2n^2 = O(n^3)$

Idea: First select a value of n_0 and see whether a value of C is found for which $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$.

Assume $n_0 = 1$, then c must be ≥ 2

Assume $n_0 = 2$, then c must be ≥ 1

Big “oh” notation: example

Example: Show that $f(n) = n^2 + 2n + 1$ is $O(n^2)$.

Solution: Since $n > 1$, then $0 \leq n^2 + 2n + 1 \leq n^2 + 2n^2 + n^2 = 4n^2$
We need to prove that there exists a pair (c, n_0) such that $4n^2 \leq cn^2$
for all $n \geq n_0$

Assume that $n_0 = 1$, then c must be ≥ 4

Assume that $n_0 = 2$, then c must be ≥ 4

Example: Show that $7n^2$ is $O(n^3)$.

Solution:

We need to prove that there exists a pair (c, n_0) such that $7n^2 \leq cn^3$
for all $n \geq n_0$

Assume that $n_0 = 1$, then c must be ≥ 7

Big “oh” notation: example

Example: Show that n^2 is not $O(n)$.

Solution: To show that n^2 is not $O(n)$, we must show that no pair (c, n_0) exist such that $n^2 \leq cn$ whenever $n \geq n_0$.

- We will use a proof by contradiction.
- Suppose that a (c, n_0) exist that satisfies the inequality.
- Observe that when $n > 0$ we can divide both sides of the inequality $n^2 \leq cn$ by n , we get $n \leq c$.
- However, no matter what c and n_0 are, the inequality $n \leq c$ cannot hold for all n with $n \geq n_0$.
- In particular, once we set a value of n_0 , we see that when n is larger than the maximum of c and n_0 , the inequality fails.

Example: Is it also true whether n^3 is $O(7x^2)$.

H.W.

Big “oh” estimates: some functions

Theorem: Let $f(x) = a^n x^n + a^{n-1} x^{n-1} + \dots + a_1 x + a_0$, where $a_0, a_1, \dots, a_{n-1}, a_n$ are real numbers. Then $f(x)$ is $O(x^n)$.

Example: Big“oh” estimate of first n positive integers.

Solution: $1 + 2 + \dots + n \leq n + n + \dots + n = n^2$.

So, $1 + 2 + \dots + n = n^2$

Example: Big“oh” estimate of $n!$.

Solution: $n! = 1 \cdot 2 \cdot \dots \cdot n \leq n \cdot n \cdot \dots \cdot n = n^n$.

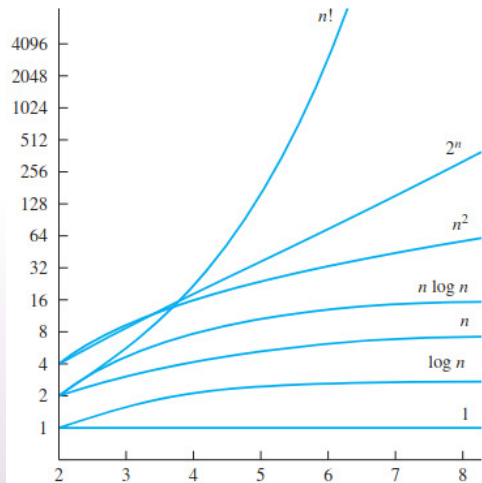
So $n! = O(n^n)$

$\log n! \leq \log n^n = n \log n$.

This implies that $\log n!$ is $O(n \log n)$,

Order of the functions

1 , $\log n$, n , $n \log n$, n^2 , 2^n , $n!$



Growth of combinations of functions

Theorem: $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$. Then $(f_1 + f_2)(x)$ is $O(g(x))$, where $g(x) = \max\{g_1(x), g_2(x)\}$ for all x .

Corollary: $f_1(x)$ and $f_2(x)$ are both $O(g(x))$. Then $(f_1 + f_2)(x)$ is $O(g(x))$.

Theorem: $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$. Then $(f_1 \cdot f_2)(x)$ is $O(g_1(x) \cdot g_2(x))$.

Examples

Example: Give a big “oh” estimate for $f(n) = 3n \log(n!) + (n^2 + 3) \log n$, where n is a positive integer.

Solution:

- $3n \log(n!) = 3n \cdot O(n \log n) = O(n^2 \log n)$.
- $3n \log(n!) + (n^2 + 3) \log n = O(n^2 \log n) + [O(n^2) \cdot \log n] = O(n^2 \log n)$
as $(n^2 + 3) = O(n^2)$

Example: Give a big “oh” estimate for $f(x) = (x + 1) \log(x^2 + 1) + 3x^2$.

Solution:

- $(x + 1)$ is $O(x)$
- $x^2 + 1 \leq 2x^2$ when $x > 1$.
- $\log(x^2 + 1) \leq \log(2x^2) = \log 2 + \log x^2 = \log 2 + 2 \log x \leq 3 \log x$
- $(x + 1) \log(x^2 + 1) = O(x) \cdot O(\log x) = O(x \log x)$
- $(x + 1) \log(x^2 + 1) + 3x^2 = O(x \log x) + O(x^2) = O(x^2)$

Big “omega” notation (lower bound)

Let $f(n)$ be a function defined on $n = 1, 2, 3, \dots$

$f(n) = \Omega(g(n))$ if there exist constants $c > 0$, $n_0 > 0$ such that

$$0 \leq cg(n) \leq f(n) \quad \text{for all } n \geq n_0$$

Example: Is $5n^2 + 3n = \Omega(n^2)$

Solution:

We need to prove that there exists a pair (c, n_0) such that $5n^2 + 3n \geq cn^2$ for all $n \geq n_0$

Assume $n_0 = 1$, then c must be ≤ 5

“thete” notation (tight bound)

Let $f(n)$ be a function defined on $n = 1, 2, 3, \dots$

$f(n) = \Theta(g(n))$ if there exist constants $c_1, c_2 > 0$, $n_0 > 0$ such that

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \text{for all } n \geq n_0$$

Example: Is $3n + 2 = \Theta(n)$

Solution:

Assume $n_0 = 1$, then $c_1 = 1$ and $c_2 = 6$

Small “oh” and “omega” notations

Let $f(n)$ be a function defined on $n = 1, 2, 3, \dots$

Small “oh”:

$f(n) = o(g(n))$ if there exist constants $c > 0, n_0 > 0$ such that

$$0 \leq f(n) < cg(n) \quad \text{for all } n \geq n_0$$

Small “omega”:

$f(n) = \omega(g(n))$ if there exist constants $c > 0, n_0 > 0$ such that

$$0 \leq cg(n) < f(n) \quad \text{for all } n \geq n_0$$